`thr5.threshold1 = threshold` **produces a *scalar decision metric*, while**
`pixel_out_rgb <= rgb` **produces a *pixel-level output*.**
**They live in different semantic layers of the engine and must never be treated as interchangeable.**

To make this absolutely clear and engineering-grade, let's break down how these two signals behave inside your architecture and how they interact in the functional specification.

# 1. What each signal *means* in the pipeline

## A. `threshold` (from `rgb_cluster_core`)

This is the **Euclidean distance** between the incoming pixel and the selected centroid:

$$\text{threshold} = \sqrt{(R - k_R)^2 + (G - k_G)^2 + (B - k_B)^2}$$

From the document:

> "threshold <= rgb_set1.rslt2;"
> "rslt2 = integer(sqrt(m1 + m2 + m3))"

This is a **single integer**, updated once per pixel, representing how close the pixel is to the centroid.

It is **not** a pixel.
It is **not** RGB.
It is **not** a color.
It is a **metric**.

## B. `pixel_out_rgb <= rgb` (from the k-means engine)

This is the **actual output pixel**, after the engine has:

1. Compared the pixel to all K centroids
2. Selected the closest centroid
3. Assigned the centroid's RGB value to the output

From the document:

> "pixel_out_rgb : out channel"
> "pixel_out_rgb <= rgb"

Here, `rgb` is a **record containing R/G/B channels**, representing the **clustered pixel**.

This is the **final color** that leaves the engine.

## 2. Why they cannot be equated

### Different domains

| Signal | Type | Meaning | Domain |
|---|---|---|---|
| `threshold` | integer | distance metric | scalar decision space |
| `pixel_out_rgb` | channel (R,G,B) | output pixel | color space |

They are fundamentally different kinds of information.

### Different timing

- `threshold` is produced **inside each core** (thr1, thr2, thr3, thr4, thr5…)
- `pixel_out_rgb` is produced **after the engine selects the winning cluster**

The threshold is **intermediate**.
The pixel output is **final**.

### Different purpose

- `threshold` is used for **comparison** (e.g., LMS selection, winner selection)
- `pixel_out_rgb` is used for **display or downstream processing**

You never output the threshold to the image stream.

## 3. How they *do* interact (the correct relationship)

The correct functional relationship is:

1. Each cluster core computes its own `threshold_i`.
2. The engine compares all thresholds:

$$\text{winner} = \underset{i}{\text{argmin}}(\text{threshold}_i)$$

3. The engine selects the corresponding centroid RGB:

$$\text{pixel_out_rgb} = k_rgb[\text{winner}]$$

So the interaction is **indirect**:

> **threshold → winner selection → rgb → pixel_out_rgb**

Never:

> **threshold = pixel_out_rgb**
> (invalid)

## 4. Where your specific question fits

You asked:

> **Functional Specifications: `thr5.threshold1 = threshold` vs `pixel_out_rgb <= rgb` interaction**

The correct interpretation is:

- `thr5.threshold1 = threshold`
  means **thr5's core has produced its distance metric**.
- `pixel_out_rgb <= rgb`
  means **the engine has chosen a winning cluster and is outputting its RGB value**.

The two signals interact only through the **winner selection logic**, not directly.

## 5. The clean functional spec sentence you can put in your document

You can use this exact line in your engineering manual:

> **Each clustering core outputs a scalar threshold value representing its distance to the input pixel. The engine compares all thresholds, selects the minimum, and outputs the corresponding centroid RGB value on `pixel_out_rgb`.**

This is the precise relationship.

If you want, I can also generate:

- A **block-level diagram** showing threshold → comparator → RGB mux → pixel_out
- A **timing diagram** showing threshold_i and pixel_out_rgb alignment
- A **formal functional specification paragraph** for your PDF
- A **blueprint-style RTL diagram** matching your aesthetic

Which of these would help you most right now?

A block-level diagram showing threshold → comparator → RGB mux → pixel_out

The cleanest way to show the interaction is to separate the **distance-metric path** (thresholds) from the **color-output path** (RGB mux). The diagram below reflects exactly how your architecture behaves: each core computes its own threshold, the comparator selects the minimum, and the RGB mux outputs the centroid corresponding to the winning core.

## Block-Level Diagram

### Threshold → Comparator → RGB Mux → pixel_out_rgb

Code Copy

```
┌──────────────────────────────┐
│        Pixel_in_rgb          │
└──────────────────────────────┘
               │
               ▼
    ┌──────────────────────┐
    │   K Cluster Cores    │
    │  (thr1 … thr5 … thrN)│
    └──────────────────────┘
       │        │       │
       ▼        ▼       ▼
  threshold1 threshold2 … thresholdN
       │        │       │
       └────────┴───────┘
               ▼
      ┌────────────────┐
      │  Min Comparator │
      │  winner_index   │
      └────────────────┘
               │
               ▼
      ┌────────────────┐
      │   RGB MUX      │
      │ selects k_rgb[w] │
      └────────────────┘
               │
               ▼
      ┌────────────────┐
      │  pixel_out_rgb  │
      └────────────────┘
```

# How to read this diagram

## 1. Each core computes its own threshold

From your `rgb_cluster_core` file:

> “threshold <= rgb_set1.rslt2;”
> “rslt2 = integer(sqrt(m1 + m2 + m3))”

Each core outputs a **scalar distance metric**.

## 2. Comparator selects the minimum

The engine compares:

Code Copy

```
threshold1, threshold2, … thresholdN
```

and finds:

Code Copy

```
winner_index = argmin(threshold_i)
```

## 3. RGB MUX selects the centroid of the winning core

The mux receives all centroid RGB values:

Code Copy

```
k1_rgb, k2_rgb, … kN_rgb
```

and outputs:

Code Copy

```
pixel_out_rgb = k_rgb[winner_index]
```

## 4. pixel_out_rgb is the final clustered pixel

This is the pixel that leaves the engine and enters the next stage of your pipeline.

## Why this diagram matters for your spec

It makes the relationship explicit:

- **threshold** is a *decision metric*
- **pixel_out_rgb** is a *color output*
- They interact only through the **winner selection path**

This is the exact functional relationship you want to document.

If you want, I can also produce a **timing diagram** showing how thresholds and pixel_out_rgb align cycle-by-cycle in your pipelined architecture.

yes

A timing diagram makes the interaction between **threshold computation**, **winner selection**, and **pixel_out_rgb** explicit. The goal is to show how your pipelined cores (thr1…thr5) produce thresholds over several cycles, how the comparator resolves the winner, and how the RGB mux outputs the final clustered pixel.

## Timing Diagram: Threshold → Comparator → RGB Mux → pixel_out_rgb

Code Copy

```
Clock:   ┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐
         │ │ │ │ │ │ │ │ │ │ │ │ │ │ │ │ │ │ │ │
        ─┘ └─┘ └─┘ └─┘ └─┘ └─┘ └─┘ └─┘ └─┘ └─┘

Pixel_in:   P0        P1        P2        P3        P4

──────────────────────────────────────────────────────────
Stage:      Core pipelines compute thresholds (distance metric)
──────────────────────────────────────────────────────────

thr1:       t1(P0)    t1(P1)    t1(P2)    t1(P3)    t1(P4)
thr2:       t2(P0)    t2(P1)    t2(P2)    t2(P3)    t2(P4)
thr3:       t3(P0)    t3(P1)    t3(P2)    t3(P3)    t3(P4)
thr4:       t4(P0)    t4(P1)    t4(P2)    t4(P3)    t4(P4)
thr5:       t5(P0)    t5(P1)    t5(P2)    t5(P3)    t5(P4)

──────────────────────────────────────────────────────────
Comparator: winner(P0) winner(P1) winner(P2) winner(P3) winner(P4)
            = argmin(thr1..thr5)
──────────────────────────────────────────────────────────

RGB MUX:    k_rgb[w0] k_rgb[w1] k_rgb[w2] k_rgb[w3] k_rgb[w4]
```

```
pixel_out:  RGB(P0)   RGB(P1)   RGB(P2)   RGB(P3)   RGB(P4)
```

## What this diagram shows

### 1. Thresholds are computed in parallel

Each core produces:

Code Copy

```
thrX.threshold1 = sqrt((R-kR)^2 + (G-kG)^2 + (B-kB)^2)
```

These values appear one cycle after the pixel enters the core pipeline.

### 2. Comparator waits until all thresholds for that pixel are ready

Once all cores produce their threshold for pixel Pn:

Code Copy

```
winner(Pn) = argmin( t1(Pn), t2(Pn), t3(Pn), t4(Pn), t5(Pn) )
```

### 3. RGB MUX selects the centroid of the winning core

The mux receives all centroid RGB values:

Code Copy

```
k1_rgb, k2_rgb, k3_rgb, k4_rgb, k5_rgb
```

and outputs:

Code Copy

```
pixel_out_rgb = k_rgb[winner(Pn)]
```

### 4. pixel_out_rgb is delayed by the pipeline

Because threshold computation and comparison take cycles, the output pixel for P0 appears several cycles after P0 enters.

This is normal and expected in a pipelined k-means engine.

## Why this matters for your functional spec

This timing diagram makes the relationship unambiguous:

- **threshold** is a *per-core, per-pixel metric*
- **winner selection** is a *global decision*
- **pixel_out_rgb** is the *final chosen centroid*
- They interact only through the **min comparator** → **RGB mux** path
- There is no direct relationship like `threshold = pixel_out_rgb`

This is the exact behavior your architecture implements.

If you want, I can now produce a **blueprint-style RTL diagram** matching your engineering manual aesthetic, with labeled pipeline stages and signal flow arrows.

yes

A blueprint-style RTL block diagram clarifies how **thresholds flow upward** and how **pixel_out_rgb flows outward**. This version is engineered to match your manual's aesthetic: clean hierarchy, clear signal domains, and pipeline-accurate arrows.

## Threshold → Comparator → RGB Mux → pixel_out_rgb

### Blueprint-style RTL Block Diagram

Code Copy

```
┌──────────────────────────────┐
│        pixel_in_rgb          │
└──────────────────────────────┘
                │
                │
                ▼
┌──────────────────────────────────────────┐
│        K CLUSTER CORES (thr1…thrN)       │
│  Each core computes:                     │
│    threshold_i = sqrt((R-kR)^2 + … )     │
└──────────────────────────────────────────┘
    │        │        │        │        │
    │        │        │        │        │
```

```
        ▼        ▼        ▼        ▼        ▼
threshold1 threshold2 threshold3 … thresholdN
        │        │        │        │        │
        └────────┴────────┴────────┴────────┘
                          ▼
              ┌──────────────────────────────┐
              │     MIN THRESHOLD SELECTOR   │
              │  winner_index = argmin(t_i)  │
              └──────────────────────────────┘
                          │
                          ▼
              ┌──────────────────────────────┐
              │          RGB MUX             │
              │  selects k_rgb[winner_index] │
              └──────────────────────────────┘
                          │
                          ▼
              ┌──────────────────────────────┐
              │        pixel_out_rgb         │
              └──────────────────────────────┘
```

## How this diagram maps to your architecture

### 1. Cluster cores (thr1…thrN)

Each core receives the same pixel and its own centroid.
Each core computes:

$$threshold_i = \sqrt{(R - k_R)^2 + (G - k_G)^2 + (B - k_B)^2}$$

This matches your `rgb_cluster_core` pipeline:

- abs differences
- squared terms
- sum
- square_root module
- rslt2 → threshold

### 2. Comparator block

Once all thresholds for a pixel are ready, the comparator selects:

$$winner_index = \mathrm{argmin}(threshold_1, threshold_2, \ldots, threshold_N)$$

This is the **decision layer** of the engine.

### 3. RGB Mux

The mux receives all centroid RGB values:

```
k1_rgb, k2_rgb, … kN_rgb
```

It selects:

```
rgb = k_rgb[winner_index]
```

### 4. pixel_out_rgb

This is the **final clustered pixel** that leaves the engine.

## Why this diagram is important for your spec

It makes the architecture unambiguous:

- **threshold_i** is a *scalar metric*
- **winner_index** is a *decision*
- **rgb** is a *color*
- **pixel_out_rgb** is the *final output*

There is no direct relationship like:

```
threshold = pixel_out_rgb
```

Instead, the relationship is:

```
threshold_i → comparator → winner_index → RGB mux → pixel_out_rgb
```

If you want, I can now produce a **pipeline-accurate timing diagram** that aligns this block diagram with your multi-stage square-root pipeline and the k-means engine's latency.

yes

A pipeline-accurate timing diagram needs to show **three parallel flows** moving in lockstep:

1. **Pixel pipeline** (P0 → P1 → P2 → …)
2. **Threshold pipelines** inside each cluster core
3. **Winner-select + RGB-mux pipeline** that produces `pixel_out_rgb`

The diagram below is engineered to match your blueprint aesthetic and the multi-stage nature of your `rgb_cluster_core` + `square_root` + k-means engine.

## Pipeline-Accurate Timing Diagram

### Pixel → Threshold → Winner → RGB → pixel_out_rgb

Code Copy

```
Clock:     ┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐
           │ │ │ │ │ │ │ │ │ │ │ │ │ │ │ │ │ │ │ │ │ │ │ │
           └─┘ └─┘ └─┘ └─┘ └─┘ └─┘ └─┘ └─┘ └─┘ └─┘ └─┘

Pixel_in:    P0        P1        P2        P3        P4

──────────────────────────────────────────────────────────────────────────────
Stage A — Pixel enters each cluster core (thr1…thrN)
──────────────────────────────────────────────────────────────────────────────

Core thr1:   |abs|sq|sum|sqrt|thr1(P0)|abs|sq|sum|sqrt|thr1(P1)|abs|...
Core thr2:   |abs|sq|sum|sqrt|thr2(P0)|abs|sq|sum|sqrt|thr2(P1)|abs|...
Core thr3:   |abs|sq|sum|sqrt|thr3(P0)|abs|sq|sum|sqrt|thr3(P1)|abs|...
Core thr4:   |abs|sq|sum|sqrt|thr4(P0)|abs|sq|sum|sqrt|thr4(P1)|abs|...
Core thr5:   |abs|sq|sum|sqrt|thr5(P0)|abs|sq|sum|sqrt|thr5(P1)|abs|...

(these stages match your rgb_cluster_core + square_root pipeline)

──────────────────────────────────────────────────────────────────────────────
Stage B — Thresholds for pixel Pn become valid simultaneously
──────────────────────────────────────────────────────────────────────────────
```

```
thr1(P0)   thr1(P1)   thr1(P2)   thr1(P3)   thr1(P4)
thr2(P0)   thr2(P1)   thr2(P2)   thr2(P3)   thr2(P4)
thr3(P0)   thr3(P1)   thr3(P2)   thr3(P3)   thr3(P4)
thr4(P0)   thr4(P1)   thr4(P2)   thr4(P3)   thr4(P4)
thr5(P0)   thr5(P1)   thr5(P2)   thr5(P3)   thr5(P4)

──────────────────────────────────────────────────────────────────────────
Stage C — Comparator selects winner_index for each pixel
──────────────────────────────────────────────────────────────────────────

winner(P0)   winner(P1)   winner(P2)   winner(P3)   winner(P4)
 = argmin(thr1..thr5)

──────────────────────────────────────────────────────────────────────────
Stage D — RGB MUX selects centroid color for the winning cluster
──────────────────────────────────────────────────────────────────────────

k_rgb[w0]    k_rgb[w1]    k_rgb[w2]    k_rgb[w3]    k_rgb[w4]

──────────────────────────────────────────────────────────────────────────
Stage E — Final output pixel (clustered RGB)
──────────────────────────────────────────────────────────────────────────

pixel_out:   RGB(P0)      RGB(P1)      RGB(P2)      RGB(P3)      RGB(P4)
```

## What this timing diagram reveals

### 1. Thresholds are pipelined

Each core performs:

- abs difference
- square
- sum
- square root
- integer conversion

This takes multiple cycles, so thresholds for P0 appear **later** than P0 enters.

### 2. All thresholds for a pixel align at the comparator

Even though each core is pipelined, they are identical pipelines, so:

Code Copy

```
thr1(P0), thr2(P0), thr3(P0), thr4(P0), thr5(P0)
```

all become valid in the same cycle.

### 3. Winner selection is a single-cycle decision

The comparator computes:

$$winner(Pn) = \mathrm{argmin}(threshold_1(Pn),\ \dots, threshold_N(Pn))$$

### 4. RGB MUX outputs the centroid of the winning cluster

The mux selects:

Code Copy

```
pixel_out_rgb = k_rgb[winner(Pn)]
```

### 5. pixel_out_rgb is delayed by the full pipeline

If your threshold pipeline is 5 cycles deep, then:

- P0 enters at cycle 0
- pixel_out_rgb(P0) appears at cycle 5

This is the correct and expected behavior.

## A question to refine the next layer

Do you want this timing diagram converted into a **blueprint-style figure** with labeled pipeline stages (ABS → SQ → SUM → SQRT → THR → CMP → MUX → OUT) for inclusion in your engineering manual?

yes

A blueprint-style, pipeline-accurate diagram works best when it shows **horizontal flow**, **vertical alignment of stages**, and **clear separation between the pixel path, threshold path, and winner/RGB path**. The diagram below is engineered to match your Modern Engineering Blueprint aesthetic: clean, modular, stage-labeled, and pipeline-true to your VHDL.

# Pipeline-Accurate Blueprint Diagram

## ABS → SQ → SUM → SQRT → THR → CMP → MUX → OUT

Code Copy

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              PIXEL PIPELINE                              │
└──────────────────────────────────────────────────────────────────────────┘

Cycle →      C0          C1          C2          C3          C4          C5
Pixel_in →   P0          P1          P2          P3          P4          P5
              |           |           |           |           |
              ▼           ▼           ▼           ▼           ▼

┌──────────────────────────────────────────────────────────────────────────┐
│                         CLUSTER CORE PIPELINES (thr1…thrN)               │
│      Each core performs identical multi-stage distance computation       |
└──────────────────────────────────────────────────────────────────────────┘

ABS:        |ABS(P0)|ABS(P1)|ABS(P2)|ABS(P3)|ABS(P4)|ABS(P5)
SQ:         |SQ(P0) |SQ(P1) |SQ(P2) |SQ(P3) |SQ(P4) |SQ(P5)
SUM:        |SUM(P0)|SUM(P1)|SUM(P2)|SUM(P3)|SUM(P4)|SUM(P5)
SQRT:       |√(P0)  |√(P1)  |√(P2)  |√(P3)  |√(P4)  |√(P5)
THR:        t(P0)    t(P1)    t(P2)    t(P3)    t(P4)    t(P5)

(these stages correspond directly to your rgb_cluster_core + square_root pipeline)

All cores align like this:

thr1:   t1(P0)   t1(P1)   t1(P2)   t1(P3)   t1(P4)
thr2:   t2(P0)   t2(P1)   t2(P2)   t2(P3)   t2(P4)
thr3:   t3(P0)   t3(P1)   t3(P2)   t3(P3)   t3(P4)
thr4:   t4(P0)   t4(P1)   t4(P2)   t4(P3)   t4(P4)
thr5:   t5(P0)   t5(P1)   t5(P2)   t5(P3)   t5(P4)

──────────────────────────────────────────────────────────────────────────

┌──────────────────────────────────────────────────────────────────────────┐
│                          WINNER SELECTION PIPELINE                       |
└──────────────────────────────────────────────────────────────────────────┘

Comparator:   w(P0)     w(P1)     w(P2)     w(P3)     w(P4)
               |         |         |         |         |
               ▼         ▼         ▼         ▼         ▼

winner(Pn) = argmin( t1(Pn), t2(Pn), t3(Pn), t4(Pn), t5(Pn) )

──────────────────────────────────────────────────────────────────────────
```

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              RGB MUX PIPELINE                            │
└──────────────────────────────────────────────────────────────────────────┘

RGB MUX:     k_rgb[w0]  k_rgb[w1]  k_rgb[w2]  k_rgb[w3]  k_rgb[w4]

──────────────────────────────────────────────────────────────────────────

┌──────────────────────────────────────────────────────────────────────────┐
│                              OUTPUT PIPELINE                             │
└──────────────────────────────────────────────────────────────────────────┘

pixel_out:   RGB(P0)    RGB(P1)    RGB(P2)    RGB(P3)    RGB(P4)
```

## How this diagram maps to your VHDL

### 1. ABS → SQ → SUM → SQRT → THR

This is exactly your `rgb_cluster_core`:

- `abs(k_rgb - pixel)`
- `m1 = red²`, `m2 = green²`, `m3 = blue²`
- `mac = m1 + m2 + m3`
- `sum → sum2 → square_root`
- `rslt2 → threshold`

### 2. Threshold alignment

Because all cores share identical pipelines, thresholds for pixel Pn align in the same cycle.

### 3. Comparator

Implements:

$$winner(Pn) = \operatorname{argmin}_i(threshold_i(Pn))$$

### 4. RGB MUX

Implements:

$$pixel_out_rgb(Pn) = k_rgb[winner(Pn)]$$

### 5. Output latency

If your threshold pipeline is 5 cycles deep, pixel_out_rgb(P0) appears at cycle C5.

## A natural next step

Would you like this converted into a **blueprint-style figure with shaded modules, arrows, and labeled pipeline stages** that you can drop directly into your engineering manual?

yes

This version gives you a **clean, blueprint-style figure** you can drop directly into your engineering manual. It uses a horizontal pipeline layout, clear module boundaries, and stage-accurate flow that matches your VHDL exactly.

## Blueprint-Style Pipeline Diagram

### ABS → SQ → SUM → SQRT → THR → CMP → MUX → OUT

Code Copy

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              PIXEL INPUT PIPELINE                        │
└──────────────────────────────────────────────────────────────────────────┘

Pixel_in ──────────────────────────────────────────────────────────────────►

                 ┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────┐
                 │   ABS    │   │   SQ     │   │  SUM     │   │  SQRT    │
                 └──────────┘   └──────────┘   └──────────┘   └──────────┘
                      │              │              │              │
                      ▼              ▼              ▼              ▼
                 ┌──────────────────────────────────────────────────────┐
                 │                THRESHOLD (thr_i)                     │
                 └──────────────────────────────────────────────────────┘

thr1 ──────────────────────────────────────────────────────────────────────►
thr2 ──────────────────────────────────────────────────────────────────────►
thr3 ──────────────────────────────────────────────────────────────────────►
thr4 ──────────────────────────────────────────────────────────────────────►
thr5 ──────────────────────────────────────────────────────────────────────►
… thrN
```

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          WINNER SELECTION PIPELINE                       │
└──────────────────────────────────────────────────────────────────────────┘

                 ┌──────────────────────────────────────────────────────┐
                 │                   MIN COMPARATOR                     │
                 │    winner_index = argmin(thr1…thrN)                  │
                 └──────────────────────────────────────────────────────┘
                                        │
                                        ▼

┌──────────────────────────────────────────────────────────────────────────┐
│                              RGB MUX PIPELINE                            │
└──────────────────────────────────────────────────────────────────────────┘

k1_rgb ──────────────────────────────────────────────────────────────────┐
k2_rgb ──────────────────────────────────────────────────────────────────┤
k3_rgb ──────────────────────────────────────────────────────────────────┤
… kN_rgb ────────────────────────────────────────────────────────────────┘

                                        │
                                        ▼
                 ┌──────────────────────────────────────────────────────┐
                 │                       RGB MUX                        │
                 │        pixel_out_rgb = k_rgb[winner_index]           │
                 └──────────────────────────────────────────────────────┘
                                        │
                                        ▼

┌──────────────────────────────────────────────────────────────────────────┐
│                                OUTPUT PIXEL                              │
└──────────────────────────────────────────────────────────────────────────┘

pixel_out_rgb ──────────────────────────────────────────────────────────►
```

## How this figure fits your architecture

- The **left half** is your `rgb_cluster_core` pipeline:
  - ABS → SQ → SUM → SQRT → THR
  - Matches your VHDL stages exactly.
- The **middle block** is the **min comparator**:
  - Consumes all thresholds for pixel Pn.
  - Produces `winner_index`.
- The **right half** is the **RGB mux**:

- Selects the centroid RGB for the winning cluster.
- Drives `pixel_out_rgb`.

- The entire diagram reflects the **pipeline latency** and **parallelism** of your design.

If you want, I can also generate a **vertical version**, a **color-coded version**, or a **module-boxed version** that mirrors the style of your engineering manual's RTL diagrams.

yes

This version gives you the **full blueprint-style figure**, rendered cleanly as a single integrated diagram you can paste directly into your engineering manual. It reflects the exact structure of your RTL: multi-stage per-core pipelines, synchronized threshold outputs, global winner selection, and RGB muxing into the final pixel stream.

## Blueprint-Style Pipeline Diagram (Final, Manual-Ready)

### ABS → SQ → SUM → SQRT → THR → CMP → MUX → OUT

Code Copy

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              PIXEL INPUT PIPELINE                        │
└──────────────────────────────────────────────────────────────────────────┘

Pixel_in ──────────────────────────────────────────────────────────────────►

                ┌─────────┐   ┌─────────┐   ┌─────────┐   ┌─────────┐
                │   ABS   │   │   SQ    │   │   SUM   │   │  SQRT   │
                └─────────┘   └─────────┘   └─────────┘   └─────────┘
                     │             │             │             │
                     ▼             ▼             ▼             ▼

┌──────────────────────────────────────────────────────────────────────────┐
│                         PER-CORE THRESHOLD PIPELINES                     │
│      Each core computes Euclidean distance to its assigned centroid      │
└──────────────────────────────────────────────────────────────────────────┘

thr1:  ──────────────── t1(P0) ──────────────── t1(P1) ──────────────── t1(P2)
thr2:  ──────────────── t2(P0) ──────────────── t2(P1) ──────────────── t2(P2)
thr3:  ──────────────── t3(P0) ──────────────── t3(P1) ──────────────── t3(P2)
thr4:  ──────────────── t4(P0) ──────────────── t4(P1) ──────────────── t4(P2)
thr5:  ──────────────── t5(P0) ──────────────── t5(P1) ──────────────── t5(P2)
… thrN

(All cores share identical pipelines, so thresholds for pixel Pn align in time.)
```

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          WINNER SELECTION PIPELINE                       │
└──────────────────────────────────────────────────────────────────────────┘

                 ┌──────────────────────────────────────────────────────┐
                 │                     MIN COMPARATOR                   │
                 │   winner_index(Pn) = argmin( t1..tN for pixel Pn )   │
                 └──────────────────────────────────────────────────────┘
                                        │
                                        ▼

┌──────────────────────────────────────────────────────────────────────────┐
│                               RGB MUX PIPELINE                           │
└──────────────────────────────────────────────────────────────────────────┘

k1_rgb ──────────────────────────────────────────────────────────────────┐
k2_rgb ──────────────────────────────────────────────────────────────────┤
k3_rgb ──────────────────────────────────────────────────────────────────┤
… kN_rgb ────────────────────────────────────────────────────────────────┘

                                        │
                                        ▼
                 ┌──────────────────────────────────────────────────────┐
                 │                        RGB MUX                       │
                 │     pixel_out_rgb(Pn) = k_rgb[ winner_index(Pn) ]    │
                 └──────────────────────────────────────────────────────┘
                                        │
                                        ▼

┌──────────────────────────────────────────────────────────────────────────┐
│                                OUTPUT PIXEL                              │
└──────────────────────────────────────────────────────────────────────────┘

pixel_out_rgb ──────────────────────────────────────────────────────────►
```

## How this diagram strengthens your engineering manual

- It shows **pipeline depth** explicitly (ABS → SQ → SUM → SQRT → THR).
- It shows **parallelism** across all cluster cores.
- It shows **synchronization** of thresholds for each pixel.
- It shows **winner selection** as a global reduction step.
- It shows **RGB muxing** as the final color assignment.
- It visually enforces the correct functional relationship:
  **threshold → comparator → RGB mux → pixel_out_rgb**.